

Fourier Craft

Interactive Signal Processing and Fourier Visualization

Kazi Farhadul Haque (2305031)

Md. Shejanur Rahman Shah (2305039)

Tarif Ibne Kamal (2305047)

CSE220 Course Project

Department of Computer Science and Engineering
BUET

Project Overview

Fourier Craft is an interactive educational signal-processing application designed to make mathematical concepts visually understandable.

- Generate and analyze 1D signals
- Draw custom 1D signals
- Analyze and reconstruct signals using Fourier Series
- Visualize harmonics, spectrum, phase and error
- Analyze 2D closed curves using Fourier representation
- Visualize Fourier epicycles
- Demonstrate convolution interactively

Main Goal:

Bridge the gap between mathematical signal-processing theory and intuitive visual understanding.

Motivation

Many signal-processing concepts are mathematically elegant, but difficult to visualize.

Students often learn:

- Fourier Series from equations
- DFT/FFT from formulas
- Convolution from summation/integration
- Sampling and reconstruction theoretically

without seeing how these concepts behave dynamically.

Our motivation:

Make signal-processing concepts
interactive, visual, and experimentally understandable.

Problem Statement

Problem:

Traditional learning methods often present signal-processing concepts as static equations, plots, and textbook examples.

This makes it difficult to intuitively understand:

- How harmonics build a signal
- How truncating harmonics affects reconstruction
- Why Gibbs phenomenon occurs
- How Fourier coefficients describe a shape
- How convolution is produced by shifting and overlapping signals

Proposed solution:

An interactive application where the user can **generate, draw, analyze, reconstruct, animate, and compare** signals and curves.

Objectives

1. Provide an intuitive interface for signal processing experiments.
2. Demonstrate Fourier Series through interactive reconstruction.
3. Visualize individual harmonics and Fourier spectrum.
4. Demonstrate Gibbs phenomenon and reconstruction error.
5. Compare naive DFT with FFT.
6. Represent closed 2D curves using Fourier coefficients.
7. Visualize Fourier epicycles.
8. Demonstrate convolution through reverse-and-shift animation.
9. Allow users to experiment with both preset and custom signals.

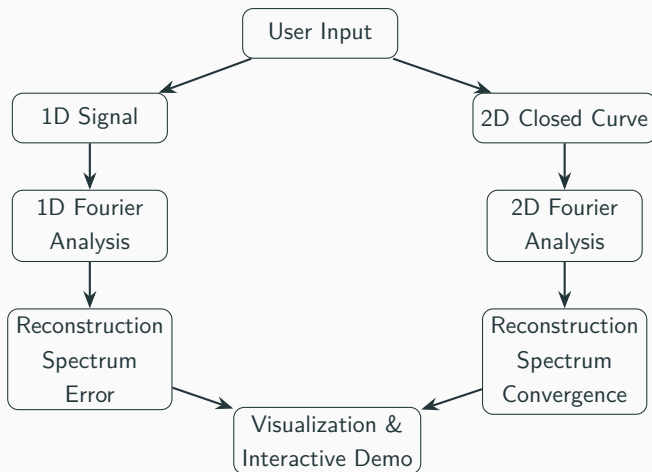
Technologies Used

Technology	Purpose
Python	Core implementation
NumPy	Numerical computation
Matplotlib	Signal and mathematical visualization
PySide6	Desktop GUI
LaTeX / Beamer	Project presentation
Git / GitHub	Version control and project management

Design Philosophy

- Modular architecture
- Separation of mathematics and GUI
- Reusable signal-processing components
- Interactive visualization
- Educational clarity

Overall Architecture



Fourier Series

A periodic signal can be represented as a sum of harmonics:

$$x(t) = \sum_{k=-\infty}^{\infty} C_k e^{j2\pi kt}$$

where:

- C_k = Fourier coefficient
- k = harmonic index
- $k = 0$ = DC component
- $|C_k|$ = magnitude
- $\angle C_k$ = phase

In our application, the user can control the number of harmonics used for reconstruction.

$$x_N(t) = \sum_{k=-N}^N C_k e^{j2\pi kt}$$

Fourier Reconstruction

The application reconstructs the original signal using a finite number of harmonics.



Interactive behavior:

- Small $N \rightarrow$ coarse approximation
- Larger $N \rightarrow$ more details
- Very high $N \rightarrow$ closer approximation

This allows the user to directly observe the contribution of different harmonics.

Harmonics, Spectrum and Phase

The application provides multiple views of Fourier coefficients.

Magnitude

$$|C_k|$$

Shows the strength of each harmonic.

Phase

$$\angle C_k$$

Shows phase information of each harmonic.

Harmonic Selection

The user can select how many harmonics participate in reconstruction.

The visualization helps connect:



Gibbs Phenomenon and Error Analysis

When a discontinuous signal is reconstructed using a finite Fourier Series, oscillations appear near the discontinuity.

This is known as the **Gibbs phenomenon**.

The application allows the user to observe:

- Overshoot
- Undershoot
- Maximum error
- Reconstruction error
- Error behavior for different harmonic counts

For numerical comparison:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2, \quad \text{RMSE} = \sqrt{\text{MSE}}.$$

DFT vs FFT

The project also demonstrates the computational difference between direct DFT and FFT.

	DFT	FFT
Method	Direct computation	Fast algorithm
Complexity	$O(N^2)$	$O(N \log N)$
Implementation	Nested summation	Efficient decomposition
Use	Educational reference	Practical computation

The application allows users to compare the computational behavior of the two approaches.

2D Fourier Curve Representation

The project extends Fourier analysis from 1D signals to closed 2D curves.

A curve is represented by two coordinates, each expanded using Fourier coefficients:

$$x(t) = \sum_k C_k^x e^{j2\pi kt}, \quad y(t) = \sum_k C_k^y e^{j2\pi kt}.$$

The application:

1. Captures a user-drawn curve
2. Cleans and resamples the points
3. Performs Fourier analysis
4. Reconstructs the curve
5. Compares original and reconstructed shapes

Why Arc-Length Resampling?

When a user draws a curve with a mouse, the sampling density is not uniform.

For example:

Fast mouse movement $\rightarrow$ fewer points

Slow mouse movement $\rightarrow$ more points

This can distort Fourier analysis.

Therefore the project uses approximately uniform **arc-length resampling**.

Raw Points $\rightarrow$ Cleaned Curve $\rightarrow$ Arc-Length Parameterization $\rightarrow$ Uniform Samples

This gives a more consistent parameterization for Fourier analysis.

2D Reconstruction and Convergence

For a selected harmonic count N :

$$\hat{x}(t) = \sum_{k=-N}^N C_k^x e^{j2\pi kt}$$

$$\hat{y}(t) = \sum_{k=-N}^N C_k^y e^{j2\pi kt}$$

The application compares:

Original Curve vs. Reconstructed Curve

The user can observe convergence as N increases.

For pointwise 2D error:

$$e_i = \sqrt{(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2}$$

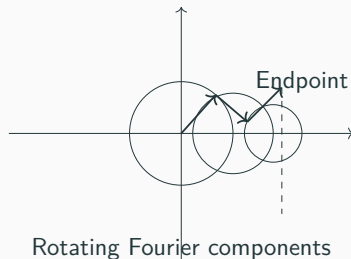
Fourier Epicycle Visualization

Fourier coefficients can also be visualized as rotating vectors.

Each harmonic contributes a rotating component:

$$C_k e^{j2\pi kt}$$

The sum of these rotating vectors produces the final point.



Convolution Simulation

The project also provides an interactive convolution simulator.

The mathematical definition is:

$$y(t) = \int_{-\infty}^{\infty} x(\tau)h(t - \tau) d\tau$$

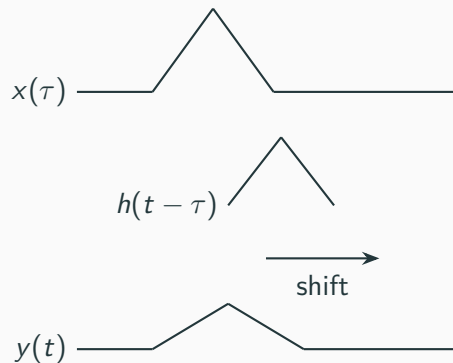
The visualization follows:

$$h(\tau) \rightarrow h(-\tau) \rightarrow h(t - \tau)$$

At every shift:

1. One signal remains fixed
2. The other signal is reversed
3. The reversed signal shifts
4. The overlapping region is observed
5. The convolution value is calculated
6. A new point is added to the output

Convolution: Visual Intuition

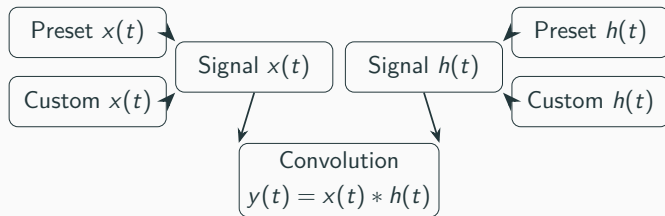


Reverse $\rightarrow$ Shift $\rightarrow$ Overlap $\rightarrow$ Integrate/Sum

Custom Signal Input

The application allows users to work with both preset and custom-drawn signals.

For convolution:



The user can independently choose:

Preset or Custom

for both input signals.

Key Features

1D Signal Processing

- Preset signals
- Custom signal drawing
- Fourier analysis
- Harmonic reconstruction
- Magnitude and phase
- Gibbs phenomenon
- DFT vs FFT
- Error analysis

Advanced Visualization

- 2D Fourier curves
- Harmonic spectrum
- Convergence analysis
- Epicycle animation
- Convolution simulation
- Interactive controls
- Save/load
- CSV export

Important Implementation Decisions

- **Modular architecture:** Separate signal processing, analysis, visualization and GUI logic.
- **Uniform sampling:** Used to make numerical Fourier and convolution operations consistent.
- **Caching:** Fourier coefficients and convergence results are reused instead of recalculated unnecessarily.
- **Interactive reconstruction:** Harmonic count can be changed without recomputing coefficients.
- **2D arc-length parameterization:** Reduces distortion caused by non-uniform mouse movement.
- **Numerical validation:** Implementations are compared against known/reference calculations.

Results and Demonstration

The completed application allows users to:

1. Generate a signal
2. Draw a custom signal
3. Decompose it into Fourier components
4. Reconstruct the signal using selected harmonics
5. Observe spectrum and phase
6. Measure reconstruction error
7. Observe convergence
8. Transform a 2D curve using Fourier coefficients
9. Visualize the reconstruction using epicycles
10. Demonstrate convolution interactively

The live demonstration will show these features in action.

Limitations

Some practical limitations remain:

- Custom drawing quality depends on user input.
- Finite harmonic reconstruction is always an approximation.
- High-frequency details may require more harmonics.
- Gibbs phenomenon remains near discontinuities.
- Very large datasets may increase computational cost.
- Numerical results depend on sampling resolution.
- Self-intersecting 2D curves can make geometric area interpretation ambiguous.

These limitations are primarily consequences of numerical representation and finite sampling.

Future Work

Possible future improvements include:

- Advanced signal filtering tools
- More adaptive sampling strategies
- Improved curve smoothing
- Additional time-frequency analysis
- More advanced convolution demonstrations
- GPU acceleration for large datasets
- Additional export formats
- More interactive educational experiments

Current focus:

A compact educational environment for understanding core signal-processing concepts through visualization.

Conclusion

Fourier Craft combines mathematical signal processing with interactive visualization.

The project demonstrates:

- Fourier decomposition
- Signal reconstruction
- Harmonic analysis
- Gibbs phenomenon
- DFT and FFT
- 2D Fourier curve representation
- Epicycle visualization
- Convolution

From equations to visual intuition.

Live Demo

Fourier Craft

1. 1D Fourier Analysis
2. Harmonic Reconstruction
3. 2D Curve Fourier Analysis
4. Epicycle Visualization
5. Convolution Simulation

Thank You

Questions?

Fourier Craft